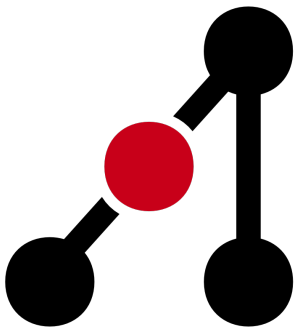
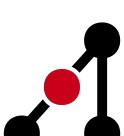


Adelaide Competitive Programming Club

Round 2 Workshop

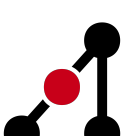
Complexity, Big O Notation and Basic Algorithms & Data Structures

Complexity and Big O



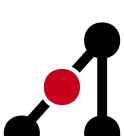
Complexity

- The performance of a program may vary between different systems.
  - Even on the same system, a program is likely to vary in speed and memory usage every time it is executed.
- Complexity is a theoretical measure of a program's execution time as its input size increases.
- Complexity analysis gives us a marker that allows for the comparison of programs and algorithms, regardless of the machine it runs on.



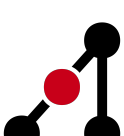
Time and Space Complexity

- We apply complexity in both time and space dimensions.
- Time complexity represents the theoretical runtime of our algorithm.
- Space complexity represents the memory usage of our program.



How Can We Measure Time and Space Complexity?

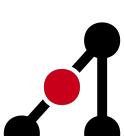
- The **Big O** notation is the mathematical representation of time and space complexity.
- We represent Big O notation using $O(n)$, where `n` is the size of the input we are operating on.
- Big O represents an upper-bound for the complexity of our program, meaning the runtime will not exceed this expression.
- The expression within $O(...)$ is the rate of growth at which our program's complexity will scale.
- For example:
  - We say that $O(n)$ time complexity is a program which runs in *linear time*.
  - We say that $O(n^2)$ time complexity is a program which runs in *quadratic time*.

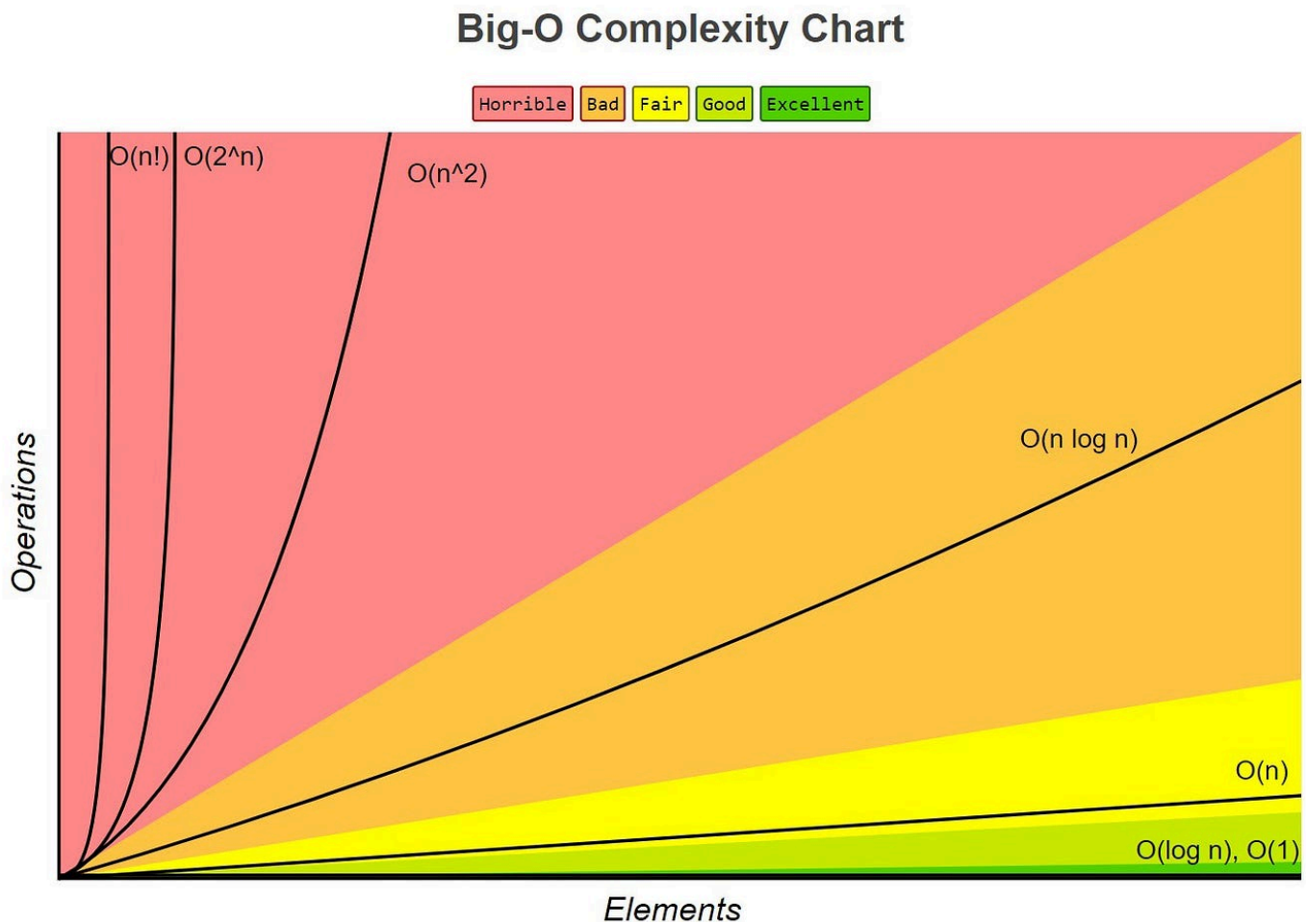


What Complexity Are We Allowed in Competitive Programming?

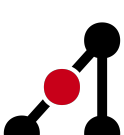
- Online judges iterate about 10^7 times per second.
- Code is generally given 1 or 2 seconds to complete.
- This means we have approximately $1 * 10^7$ to $2 * 10^7$ max program iterations before an algorithm will fail due to exceeding the time limit.

- For example, say $n = 10^5$:
  - Algorithm A executes with a time complexity of $O(n)$, 10^5 operations.
    - $\therefore$ Operations are executed within the time limit: $10^5 < 2 * 10^7$
  - Algorithm B executes with a time complexity of $O(n^2)$, 10^{10} operations.
    - $\therefore$ Time limit exceeded (TLE): $n = 10^{10} < 2 * 10^7$




Big-O Complexity Chart
Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations
Elements

Complexity Analysis

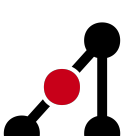


Steps for Analysis

1. Identify input size n: number of elements, nodes, or other units that scale.
2. Count basic operations: loops, recursive calls, and lookups.
3. Express as a function of n:

- For example: $O(n)$, $O(n^2)$, $O(\log n)$.

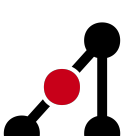
4. Keep the dominant term and drop constants.



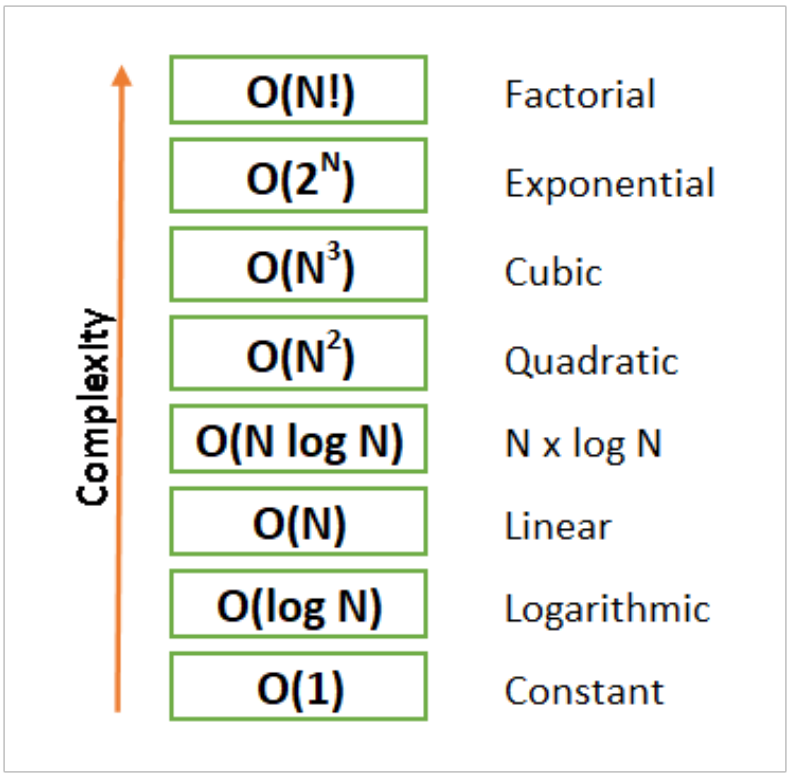
Dominance in Complexity

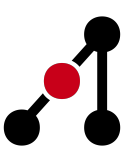
- The dominant term in a complexity analysis is that which grows fastest.
- Other terms which are not dominant, will eventually become negligible in size as n approaches infinity (you will learn more about this in math or algorithms classes).
- Note that constant terms, denoted $O(1)$ can be ignored for sufficiently small constant values.

- $f(n) = 3n^2 + 5n + 7$, Dominant term is n^2, so complexity is $O(n^2)$
- $g(n) = n \log n + 1000$, Dominant term is $n \log n$, so complexity is $O(n \log n)$
- $h(n) = 50n + 200$, Dominant term is n, so complexity is $O(n)$
- $i(n) = 50 + 200$, Dominant term is constant, so complexity is $O(1)$



Order of Complexity Dominance

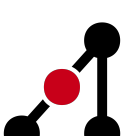




$O(1)$ Complexity Example

```python
def get_first(arr):
  return arr[0]
```

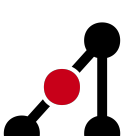
- This operation to access a random element within an array can be done in constant time.
- With an index, we travel travel directly to this memory location and return the variable stored here.



$O(n)$ Complexity Example

```python
def sum_array(arr):
    total = 0
    for x in arr:          # O(n)
        total += x         # O(1)
    return total
```

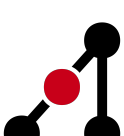
- We iterate over the entire array of size n.
  - At each iteration we are performing basic arithmetic operations, which can be done in constant time, $O(1)$.
- We can calculate the complexity looking at the nested operations and multiplying their complexities together:
  - $O(n) * O(1) = O(n * 1) = O(n)$



$O(n^2)$ Complexity Example

```python
def print_pairs(arr):
    for i in range(len(arr)):          # O(n)
        for j in range(len(arr)):      # O(n)
            print(arr[j])              # O(1)
```

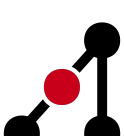
- We iterate over the entire array of size n.
  - But for each outer loop, we loop through the entire array of size n again.
    - At each iteration we are printing a single number, which can be done in constant time, $O(1)$.
- We can calculate the complexity looking at the nested operations and multiplying their complexities together:
  - $O(n) * O(n) * O(1) = O(n * n * 1) = O(n^2)$



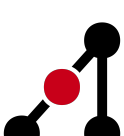
$O(n * m)$ Complexity Example

```python
def process(nums, s):
    # nums has size n
    # string, s, has size m
    for x in nums:          # O(n)
        for c in s:         # O(m)
            print(x, c)     # O(1)
```

- We iterate over the entire array of size n.
  - For each iteration of the array, we iterate over the whole string of size m
    - For each character in the string, we print the current index of the array and character of the string we are looking at.
- We can calculate the complexity looking at the nested operations and multiplying their complexities together:
  - $O(n) * O(m) * O(1) = O(n * m * 1) = O(n * m)$



Sorting and Searching



Sorting

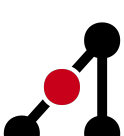
Sacrificing the upfront cost of sorting, which is typically to the order of $O(n \log n)$, allows us to operate on the data more efficiently, since we know the order of the elements.

In Python we sort by:

```python
nums = [4,5,3,1,2]
newList = sorted(nums);                 # Put sorted list into newList = [1,2,3,4,5]
nums.sort();                            # Sort in place: nums = [1,2,3,4,5]
```

In C++ we sort by:

```cpp
vector<int> nums = {4,5,3,1,2};
sort(nums.begin(), nums.end());         // Inplace
ranges::sort(nums);                     // Inplace
```

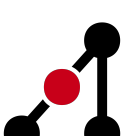


Naive Searching

- What if we want to check to see if an element exists in an array?
- We could iterate across all of the elements from left to right and check our array.

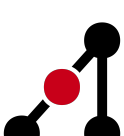
```python
def linear_search(arr, target):
  for i in range(len(arr)):
      if arr[i] == target:
          return i                # return index if found
  return -1                       # not found
```

- The complexity here is $O(n)$ because we are iterating across the entire array.

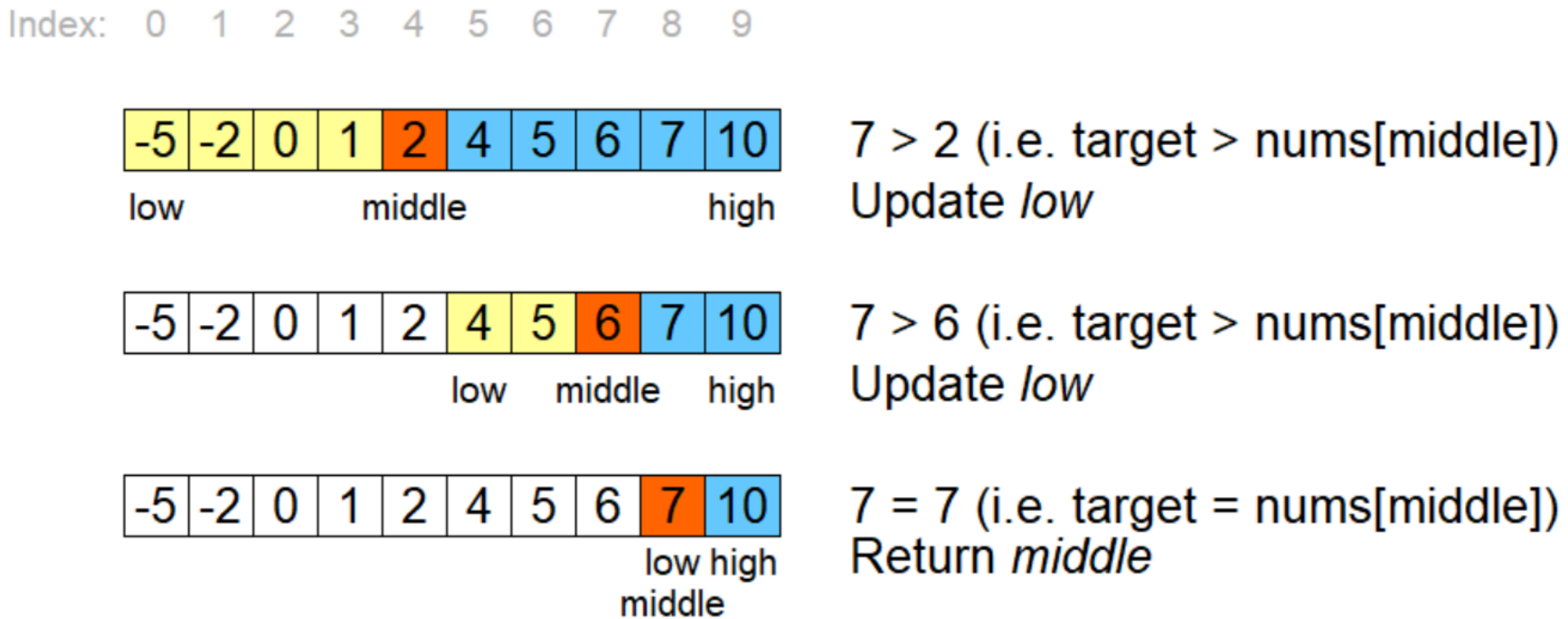


Binary Search

- What if our array is sorted?
- We could look at the middle value first.
  - If this middle value is too small, we should look to the right.
  - Else, if this middle value is too large, we should look to the left.
  - Else, the middle value is equal to our target so we have found it!



Binary Search Example

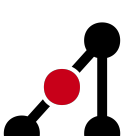


Binary Search Complexity

- Binary search can be performed in $O(\log n)$, which is far more efficient that $O(n)$.
- Why?

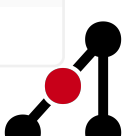
1. After the first step we have $\frac{n}{2}$ elements remaining in the search space.
2. After the first step we have $\frac{n}{4}$ elements remaining in the search space.
3. After the first step we have $\frac{n}{8}$ elements remaining in the search space.

- We can say that at any binary search step, k, the remaining search size will be = $\frac{n}{2^k}$
- We stop when $\frac{n}{2^k} = 1$, when one element remains we must have either found our target or it does not exist in the array.
- Therefore, $n = 2^k$ so $k = \log_2(n)$ iterations are required to find an element.
- Our complexity of the binary search is $O(\log_2 n) \sim O(\log n)$.



Binary Search Code

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1        # start with 2 pointers at the edges

    while left <= right:
        mid = (left + right) // 2        # calculate the midpoint index

        if arr[mid] == target:           # if we find the target, return its index
            return mid
        elif arr[mid] < target:          # current number is too small
            left = mid + 1
        else:                            # current number is too large
            right = mid - 1

    return -1                            # failed to find the index, return sentinel
```

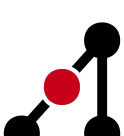


Binary Search Questions

Some binary search questions on leetcode! We don't expect you to kno everything so make sure to ask us for help.

- 704. Binary Search
- 69. Sqrt(x)

2 Pointers

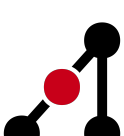


2 Pointers

- Did anyone notice the use of two pointers to track our search space in the binary search?

```python
def binary_search(arr, target):
  left, right = 0, len(arr) - 1         # start with 2 pointers at the edges
```

- Using two pointers is a powerful tool in navigating data structures.
- We organise two pointers in many ways:
  - We could have them travelling from opposite ends of an array, converging in the centre.
  - We could use them to simulate a sliding window.
  - We could have a slow pointer and fast pointer (detecting cycles), advanced.



2 Pointers Travelling from Opposite Ends Examplpe

- How can we check that a word is palindrome?
  - Palindrome means it reads the same right to left as it does left to right.

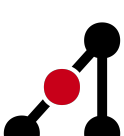
```python
def is_palindrome(s):
  left, right = 0, len(s) - 1
  while left < right:
      if s[left] != s[right]:
          return False
      left += 1
      right -= 1
  return True

print(is_palindrome("racecar"))  # True
print(is_palindrome("hello"))    # False
```

2 Pointers Sliding Window

- How can we find the maximum subarray sum of an array?
- Which subarray of size k has the largest sum of its numbers?
- Naively:

```python
def max_sum_subarray_naive(nums, k):
    max_sum = float('-inf')
    for i in range(len(nums) - k + 1):
        current_sum = 0
        for j in range(i, i + k):
            current_sum += nums[j]
        max_sum = max(max_sum, current_sum)
    return max_sum
```

- Takes $O(n * k)$ to complete.

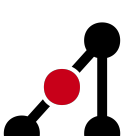


2 Pointers Sliding Window

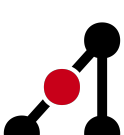
- Optimised:

```python
def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k])
    max_sum = window_sum

    for i in range(k, len(nums)):
        window_sum += nums[i]                    # take in the newest element
        window_sum -= nums[i - k]                # kick out the oldest element
        max_sum = max(max_sum, window_sum)

    return max_sum
```

- Takes $O(n)$ to complete as we only add or subtract each element once, $O(2n) = O(n)$.



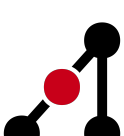
Prefix Sum



Prefix Sum

- A prefix sum allows use quickly check the sum of a range of numbers in constant time.
  - I.e. `prefix[i]` represents the sum of numbers from index `0` to `i`.
  - I.e. `prefix[i] - prefix[i-k]` represents the sum of numbers from index `i-k+1` to `i`.
- This is useful if you are asked for the sum of numbers in a range, numerous times.
- By spending $O(n)$ time and space to build a prefix sum, you can answer range sum queries in $O(1)$ time:

```python
def range_sum(prefix, L, R):
  if L == 0:
      return prefix[R]
  return prefix[R] - prefix[L - 1]
```



Building the Prefix Sum

```python
def build_prefix(nums):
    prefix = [0] * len(nums)

    prefix[0] = nums[0]

    for i in range(1, len(nums)):
        prefix[i] = prefix[i - 1] + nums[i]

    return prefix
```

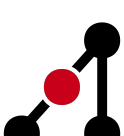
- You can build a *suffix* sum by constructing the prefix sum backwards, from right to left.

Prefix Sum Practise Questions

Here are some prefix sum question to deepen your understanding!

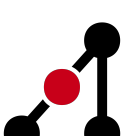
- 1480. Running Sum of 1D Array
- 2574. Left and Right Sum

Greedy



What is Greedy

- A greedy algorithm builds a solution step-by-step, always choosing the locally optimal choice at each step.
- It does not reconsider previous choices, unlike backtracking.
- Works best when the problem has:
  - Greedy choice property: local optimum leads to global optimum.
  - Optimal substructure: optimal solution can be built from optimal subproblems.
- Typically used for:
  - Scheduling problems
  - MSTs (Prim's, Kruskal's) (more on this in future workshops or algorithms classes)
  - Shortest paths (Dijkstra's)
  - Interval problems

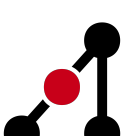


Greedy Example

- Coin change: given a set of coins (with replacement), find the minimum amount of coins required to reach a target sum.

```python
def coin_change_greedy(amount):
    coins = [25, 10, 5, 1]
    count = 0

    for coin in coins:
        while amount >= coin:
            amount -= coin
            count += 1
    return count
```

- Having the coins in sorted order allow us to use the greedy strategy, where we trial the next largest coin if the current coin is to large.



Complexity Quiz Time